



How to Run GenomelQ in VS Code

A copy-paste-friendly guide for running the project from inside Visual Studio Code.

1. Open the project

1. Launch VS Code.
2. **File** → **Open Folder...** → choose the `dna-disease-classifier` folder.
3. The first time you open it, VS Code may ask "Do you trust this folder?" — click **Yes, I trust the authors**.

2. Install the recommended extensions

When you open the folder, VS Code will prompt you to install the recommended extensions (defined in `.vscode/extensions.json`).

If the prompt doesn't appear:

1. Press **Ctrl+Shift+X** to open the Extensions panel.
2. In the search box type `@recommended`.
3. Install everything under "Workspace Recommendations":
 - **Python** (Microsoft)
 - **Pylance**
 - **Python Debugger** (debugpy)
 - **Jupyter** (optional, for `.ipynb` files)
 - **Even Better TOML, YAML** (helpers)

3. Pick the Python interpreter

The project is configured to use the existing `venv/`.

1. Press **Ctrl+Shift+P** → type **"Python: Select Interpreter"** → Enter.
2. Pick the entry that ends with `venv\Scripts\python.exe` (it should already be highlighted because of `.vscode/settings.json`).
3. Bottom-right of the status bar should now read `Python 3.10.x ('venv': venv)`.

If the `venv` interpreter is not in the list:

```
# In a VS Code integrated terminal (Ctrl+`)
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
pip install torch --index-url https://download.pytorch.org/whl/cu124 # GPU only
```

Then re-run **"Python: Select Interpreter"**.

4. (Optional) Add your free Gemini key

The Copilot tab works in three modes — **demo** (offline) is enabled by default, no setup needed. To unlock the **GEMINI** mode:

1. Get a free API key from aistudio.google.com/apikey (1500 req/day, no credit card).
2. Open the file `.env` in the project root.
3. Replace the placeholder:

```
GEMINI_API_KEY=your_real_key_here
```

4. Save the file.

VS Code's debugger picks up `.env` automatically (configured in `launch.json`).

5. Run the web app (the easy way)

1. Press **Ctrl+Shift+D** to open the **Run and Debug** panel.
2. From the dropdown at the top pick  **Run GenomelQ Web App (uvicorn + reload)**.
3. Press **F5**.

VS Code will:

- Activate the venv automatically.
- Load environment variables from `.env`.
- Start uvicorn with auto-reload.
- Open `http://127.0.0.1:8000` in your default browser as soon as the server is ready.

To stop the server: click the red square on the debug toolbar at the top of the screen, or press **Shift+F5**.

6. The other one-click launchers

The same dropdown contains pre-configured runners:

Item	What it does
 Run GenomelQ Web App	Starts the FastAPI server with hot-reload + opens the browser
 Run /predict CLI on a sample sequence	Predicts a BRCA1 sample from the command line (good first smoke test)
 Run dataset audit	Diagnoses imbalance / leakage / dedup ratios
 Build augmented dataset	Cleans → dedups → augments → balances the training set
 Train TF-IDF ensemble	Retrains the main classifier (~30 seconds)
 5-fold cross-validation	Group-aware K-fold CV with full per-class report

Item	What it does
 DNABERT-2 fine-tune (3 epochs)	Fine-tunes the transformer (~6 minutes on RTX 3050)

For each: choose it in the dropdown and press **F5**.

7. Set breakpoints

Because every launch config uses the **debugpy** debugger, you can set breakpoints anywhere in the source.

1. Click in the gutter to the left of any line number — a red dot appears.
2. Press **F5** to start the chosen run config.
3. When execution hits the breakpoint, VS Code drops into the debugger with the variable inspector, call stack, and step controls.

Use **F10** (step over), **F11** (step into), **Shift+F11** (step out), **F5** (continue).

8. Use the integrated terminal

``Ctrl+``` opens a terminal in the project root. Useful one-liners (all already wrapped as launch configs, but available manually):

```
venv\Scripts\activate

# Run the web app
uvicorn src.api.app:app --reload --port 8000

# Predict a single sequence
python -m src.predict --sequence ATGGAT...

# Predict every record in a FASTA file
python -m src.predict --file path\to\your.fasta

# Diagnose the dataset
python -m src.data.audit

# Build / rebuild the augmented training set
python -m src.data.build_dataset

# Retrain the TF-IDF ensemble
python -m src.train

# Cross-validation
python -m src.eval cv --n-splits 5

# Hyperparameter grid search
python -m src.eval grid --ks 5 6 7 --ngram-ranges 1,1 1,2 4,4

# DNABERT-2 — frozen embeddings + LR head
python -m src.ml.dnabert embed --source train
```

```
python -m src.ml.dnabert train

# DNABERT-2 - full fine-tuning
python -m src.ml.dnabert_finetune --epochs 3 --batch-size 2 --grad-accum 8
```

9. Use the Tasks runner

If you prefer `**```powershell venv\Scripts\activate`

Run the web app

```
uvicorn src.api.app:app --reload --port 8000
```

Predict a single sequence

```
python -m src.predict --sequence ATGGAT...
```

Predict every record in a FASTA file

```
python -m src.predict --file path\to\your.fasta
```

Diagnose the dataset

```
python -m src.data.audit
```

Build / rebuild the augmented training set

```
python -m src.data.build_dataset
```

Retrain the TF-IDF ensemble

```
python -m src.train
```

Cross-validation

```
python -m src.eval cv --n-splits 5
```

Hyperparameter grid search

```
python -m src.eval grid --ks 5 6 7 --ngram-ranges 1,1 1,2 4,4
```

DNABERT-2 — frozen embeddings + LR head

```
python -m src.ml.dnabert embed --source train python -m src.ml.dnabert train
```

DNABERT-2 — full fine-tuning

```
python -m src.ml.dnabert_finetune --epochs 3 --batch-size 2 --grad-accum 8
```

```
---
```

```
## 9. Use the Tasks runner
```

If you prefer `**Ctrl+Shift+B**` style task running:

- `**Ctrl+Shift+P**` → `**Tasks: Run Task**` → pick one of:
 - Install dependencies
 - Build augmented dataset
 - Train TF-IDF ensemble
 - Run web app

```
---
```

```
## 10. What you should see
```

After pressing `**F5**` with the web-app launcher:

1. The integrated terminal shows the server boot log:

INFO: Application startup complete. INFO: Uvicorn running on http://127.0.0.1:8000

2. Your default browser opens to `**http://127.0.0.1:8000**`.

3. The sidebar shows: `** Analyze ·  Copilot ·  Mutation Lab ·  Batch Upload ·  Knowledge Base ·  User Manual ·  Presentation ·  About**`.

That's it – you are running locally. Click any sample button on the `**Analyze**` page, hit `**Analyze Sequence**`, and the prediction + saliency + circular plot all light up.

```
---
```

```
## 11. Troubleshooting
```

Symptom	Likely cause	Fix
<code>`python.exe : command not found`</code>	Wrong interpreter selected	<code>`Ctrl+Shift+P`</code> → Python: Select Interpreter → choose <code>`venv\Scripts\python.exe`</code>
<code>`ModuleNotFoundError: src.api.app`</code>	Wrong working directory	Make sure VS Code

opened the ****project root****, not a sub-folder |
GPU not used during fine-tuning	Wrong torch wheel installed	Reinstall: ``pip install torch --index-url https://download.pytorch.org/whl/cu124``
Copilot says GEMINI unavailable	``.env`` missing or empty	Edit ``.env``, set ``GEMINI_API_KEY=...``, restart the server
Port 8000 already in use	Another process is running	Close the other server, or change the port: ``--port 8001``
``OSError: [WinError 10013]`` on startup	Windows blocked the port	Run VS Code as Administrator once, or pick a different port
Browser doesn't auto-open	Antivirus / popup blocker	Manually visit <http://127.0.0.1:8000>

If anything is still off, open ``models/training_metrics.json`` to confirm the trained model exists, or run ****🔄 Train TF-IDF ensemble**** to regenerate it.